

Subset Problems: Recursion States, Approaches, and Base Cases

This document summarizes the major recursion patterns and base cases used in subset problems, based on everything we've discussed.

1. Understanding the Two Main Ways to Think About Subset Problems

Almost every subset problem is solved using one of these recursion states:

Approach A: Current Sum State

State:

```
f(index, currentSum)
```

Meaning:

"I have processed elements from 0 to index-1 and currently built a subset whose sum is currentSum."

Example:

```
f(0, 0)
```

At each index:

- Take element
- Don't take element

Approach B: Remaining Target State

State:

```
f(index, target)
```

Meaning:

"Using elements from 0...index, how many ways can I make the remaining target?"

Example:

```
f(n-1, target)
```

At each index:

- Take element:

```
target -= nums[index]
```

- Don't take element:

```
target remains same
```

2. Pattern 1: Generate All Subsets

Question: Generate every possible subset.

EXAMPLE

```
nums = [1, 2, 3]
```

Output:

```
[], [1], [2], [3], [1, 2], [1, 3], [2, 3], [1, 2, 3]
```

State

```
f(index, subset)
```

Base Case

```
if(index == n){  
    ans.push_back(subset);  
    return;  
}
```

Why?

When `index == n`, every element has already been processed. Current subset becomes a valid answer.

3. Pattern 2: Check If Subset Sum Exists

Question: Is there at least one subset whose sum = target?

Return: `true / false`

State

```
f(index, currentSum)
```

or

```
f(index, target)
```

Base Case (Current Sum Version)

```
if(index == n)
    return currentSum == target;
```

EXAMPLE

nums=[1, 2, 3] , target=3

Path: Take 1, Take 2, Skip 3

Current Sum: 3

At: index==n

Return: true

Base Case (Remaining Target Version)

```
if(target == 0)
    return true;

if(index == 0)
    return nums[0] == target;
```

4. Pattern 3: Count Subsets With Sum K

Question: How many subsets have sum exactly K?

Return: count

Approach A: Current Sum State

State: f(index, currentSum)

Base Case

```
if(index == n)
    return currentSum == target;
```

Why?

At index==n all elements have been considered. Now check: Did we build target sum?

Compact Form

```
if(index == n)
    return currentSum == target;
```

because: true -> 1 and false -> 0

Approach B: Remaining Target State

State: `f(index, target)`

Standard Base Case

```
if(target == 0)
    return 1;

if(index == 0)
    return nums[0] == target;
```

Meaning

When `target == 0`, one valid way has been formed. When `index == 0`, only one element remains. Check if it can complete the target.

5. Count Subsets When Zeros Exist

This is the most important special case.

EXAMPLE

`nums=[0]`, `target=0`

Possible subsets: `{}`, `{0}`

Both produce: `sum = 0`

Answer: 2

Correct Base Case

```
if(index == 0){

    if(target == 0 && nums[0] == 0)
        return 2;

    if(target == 0 || target == nums[0])
        return 1;

    return 0;
}
```

Why Return 2?

Example: `nums=[0]`, `target=0`

Choices: Exclude 0, Include 0

Subsets: `{}`, `{0}`

Two valid subsets.

Why Return 1?

Example: `nums=[5]`, `target=0` → Only `{}` works.

Example: `nums=[5]`, `target=5` → Only `{5}` works.

6. Pattern 4: Count All Subsets

Question: Count every subset.

Formula: Every element can be taken or not taken. So: 2^n subsets.

Base Case

```
if(index == n)
    return 1;
```

Why?

Every leaf node represents one subset.

EXAMPLE

`nums=[1, 2]`

Leaves: `{}`, `{1}`, `{2}`, `{1, 2}`

Count: 4

7. Pattern 5: Partition Equal Subset Sum

Question: Can array be divided into two subsets having equal sum?

Convert to: ***target*** = ***totalSum*** / 2

Then solve: **Subset Sum Exists?**

Base Case

```
if(target == 0)
    return true;

if(index == 0)
    return nums[0] == target;
```

8. Pattern 6: Minimum Subset Sum Difference

Question: Split array into two subsets. Minimize: ***abs(S1 - S2)***

Usually solved using DP.

Base Case

```
dp[i][0] = true
```

because: Sum 0 is always possible by taking empty subset.

9. Pattern 7: Target Sum

Question: Leetcode 494 → +1 -2 +3 ... Reach target.

Convert into: **Count subsets with sum K**

Use same base case:

```
if(index == 0){  
    if(target == 0 && nums[0] == 0)  
        return 2;  
  
    if(target == 0 || target == nums[0])  
        return 1;  
  
    return 0;  
}
```

10. Early Pruning Base Case

Only when all numbers are positive.

Base Case

```
if(currentSum > target)  
    return 0;
```

Why?

If all elements are positive, current sum can only increase, never decrease.

EXAMPLE

target=10 , currentSum=12

Impossible to reach 10. Stop recursion.

11. Forward vs Backward Recursion

Forward Recursion

State: (index, currentSum)

Moves: $0 \rightarrow n-1$

Base Case: index == n

Question: What sum have I built?

Example: generate(index+1, currentSum)

Backward Recursion

State: (index, target)

Moves: $n-1 \rightarrow 0$

Base Case: index == 0

Question: Can I make this remaining target?

Example: f(index-1, target)

Quick Cheat Sheet

Problem	State	Base Case
Generate Subsets	(index, subset)	index == n
Subset Sum Exists	(index, currentSum)	currentSum == target at index == n
Subset Sum Exists DP	(index, target)	target == 0
Count Subsets Sum K	(index, currentSum)	index == n && currentSum == target
Count Subsets Sum K DP	(index, target)	index == 0 special handling
Count Subsets with Zeros	(index, target)	if(target == 0 && nums[0] == 0) return 2
Count All Subsets	(index)	return 1 at index == n
Partition Equal Subset	(index, target)	target == 0
Target Sum	(index, target)	Same as Count Subsets
Min Subset Difference	DP Table	dp[i][0] = true

RULE TO REMEMBER

Whenever solving a subset problem, first ask: "What is my recursion state?"

If state is `(index, currentSum)`, base case is usually: `index == n`

If state is `(index, target)`, base case is usually: `index == 0` or `target == 0`

The base case always depends on what information your recursion state represents, not on the problem alone. This single idea explains almost every subset-sum base case you'll encounter in recursion, memoization, and DP.